

Skylink: An ESP32-Hosted, Browser-Based Ground Control Station for ArduPilot UAVs with Dual SITL/Hardware Architecture, Firmware Deployment Verification, and Preliminary Safety Layer

Aryan Pandey, Aditya Garg, Karan Agarwal, Poojan Goyani

Department of Computer Science and Engineering, PDPM Indian Institute of Information Technology, Design & Manufacturing (IIITDM), Jabalpur, India
Supervised by Dr. Akshay Pandey

Abstract—Ground control stations (GCS) for ArduPilot-based unmanned aerial vehicles are typically hosted on laptop-class platforms requiring at least 512 MB of RAM. This paper presents Skylink, an ESP32-hosted, browser-based GCS prototype that serves a complete web interface over Wi-Fi, parses MAVLink v2 telemetry, and dispatches fifteen flight commands on a resource-constrained microcontroller (240 MHz dual-core Xtensa LX6, 327 KB SRAM) running FreeRTOS and the Arduino framework. Skylink integrates a MAVLink v2 parser and command dispatcher, an asynchronous WebSocket server delivering telemetry at 5 Hz, a Primary Flight Display (PFD) with artificial horizon and altitude tape, a Leaflet.js interactive map with live drone position tracking and click-to-fly navigation, a preflight safety checklist, and a preliminary firmware-level command-filtering layer for altitude, yaw, geofence, and command-rate constraints—with the MOVE_BODY altitude-floor defect explicitly treated as unresolved pending correction and retesting. A compile-time dual-mode design supports both ArduPilot SITL simulation (TCP) and physical Pixhawk hardware (UART): the SITL path is fully verified; hardware firmware has been deployed to and confirmed operational on a physical ESP32-WROOM-32 board, with Wi-Fi association, LittleFS-served dashboard, WebSocket communication, LED state machine, and build-version integrity all confirmed on-device; UART/Pixhawk communication and controlled flight testing remain as the immediate next steps. Five development phases with explicit acceptance criteria were completed and SITL-verified; a minimum runtime free heap of 51 KB was observed under peak SITL load and the application binary occupies 860 KB of the 1,310 KB application partition. To the authors’ knowledge, within the surveyed literature, this is the first combination of a self-hosted browser GCS, a firmware-level command-filtering layer, and a dual SITL/hardware compile-time design on a single ESP32-class device.

Index Terms—Ground control station; ESP32; MAVLink; ArduPilot; software-in-the-loop simulation; embedded systems; hardware implementation; UAV

I. INTRODUCTION

The ArduPilot ecosystem supports a wide range of deployed unmanned aerial vehicles (UAVs) across agricultural, infrastructure-inspection, and research domains [1]. Standard operator interfaces—Mission Planner [2] and QGroundControl [3]—require dedicated laptops or desktops with graphical operating systems, creating a hardware dependency that limits field deployability and imposes non-trivial cost at scale.

Web-based GCS architectures have begun decoupling operators from dedicated hardware: a browser running on any network-connected device can serve as the pilot interface, with protocol logic executing on a companion computer connected to the vehicle. Burke [4] demonstrated a browser-accessible GCS on a Raspberry Pi companion computer (512 MB+ RAM), achieving real-flight validated command and telemetry. Atwal [5] replaced a desktop tool with a Vue.js/MQTT web application backed by a dedicated PC. However, all existing web-based GCS systems require a general-purpose computer to execute protocol logic.

DroneBridge for ESP32 [7] demonstrates that the ESP32 microcontroller (327 KB SRAM, FreeRTOS, no general-purpose OS) can perform reliable MAVLink relay over Wi-Fi—but only as a transparent serial-to-Wi-Fi bridge with no self-hosted dashboard, no command semantics, and no safety filtering.

This paper presents Skylink, an ESP32-hosted, browser-based GCS prototype that closes this gap. The distinguishing contribution is combining, on the same ESP32-class device: a self-hosted single-page browser application, a full WebSocket command and telemetry layer, a MAVLink parser and dispatcher, a geofenced interactive map, a preliminary firmware-level command-filtering layer, and a dual compile-time SITL/UART transport design. No external computer, cloud subscription, or backend service is required in the target hardware configuration. As illustrated in Fig. 2, the system follows a three-tier design in which the ESP32 simultaneously serves as web server, WebSocket hub, MAVLink parser, and flight command gateway. The development followed a SITL-first methodology: all command paths were validated in ArduCopter SITL before hardware assembly, after which the hardware-mode firmware was deployed to and confirmed operational on a physical ESP32-WROOM-32 board.

The contributions of this work are:

- 1) An ESP32-hosted, browser-based GCS prototype integrating web-interface serving, WebSocket telemetry, and MAVLink command and telemetry handling on a resource-constrained microcontroller—to the authors’ knowledge, the first to combine all four elements (self-hosted browser, preliminary command-filtering layer,

dual SITL/hardware compile-time design, command dispatch) on an ESP32-class device within the surveyed literature, with firmware deployment to physical hardware confirmed.

- 2) A compile-time dual-mode transport abstraction (SITL TCP / UART hardware) enabling the complete GCS firmware to be validated against ArduPilot SITL before any physical hardware is assembled; the UART pathway is implemented in hardware-ready firmware and has been deployed to a physical ESP32-WROOM-32 board, with Pixhawk bench and flight validation as the immediate next step.
- 3) A preliminary firmware-level command-filtering layer co-located with GCS command dispatch, enforcing altitude floors, geofence radii, yaw limits, and command-rate limits independently of autopilot parameters; the AGL floor for MOVE_BODY has a known defect acknowledged in Section VII-D and is scheduled for correction before hardware flight.
- 4) A five-phase, phase-gated development methodology for embedded GCS systems with explicit acceptance criteria, demonstrated and SITL-verified across all five phases (Table V), with hardware firmware subsequently deployed to and confirmed operational on a physical ESP32 board.

II. RELATED WORK

A. Desktop GCS Applications

Mission Planner [2] and QGroundControl [3] provide comprehensive UAV ground control on Windows, Linux, and macOS. Both require installation on systems with at least 1 GB RAM; their functionality is mature but their hardware dependency is substantial.

B. Web-Based GCS Systems

Burke [4] demonstrated WebGCS, a fully AI-generated browser-accessible GCS on a Raspberry Pi companion computer, achieving arm/disarm, takeoff, land, RTL, real-time map tracking, and click-to-fly in real flight, requiring approximately 100 person-hours of development. Atwal [5] developed a Vue.js/DroneKit/MQTT web GCS demonstrating functional parity with a desktop tool, including geofence creation, but required a dedicated PC backend. The IoT-integrated GCS [6] demonstrated real-time WebSocket telemetry visualisation over 100 m flights, also requiring a PC backend. All three systems execute protocol logic on general-purpose computers (512 MB–GB class RAM); none co-locates the browser-serving, MAVLink parsing, and command-filtering layers on the same microcontroller.

C. ESP32 as MAVLink Platform

DroneBridge for ESP32 [7] implements bidirectional MAVLink relay over Wi-Fi, achieving up to 1 km range in ESP-NOW Long-Range mode with AES-GCM-256 encryption. It is a relay, not a GCS: it passes MAVLink bytes

to an external application on a PC with no command semantics, no dashboard, and no safety filtering. Separately, open-source community projects have demonstrated ESP32 firmware capable of issuing basic MAVLink commands (arm, mode change, simple waypoint). Skylink differs by integrating a complete self-hosted browser interface, a preliminary command-filtering layer, and a dual SITL/UART compile-time design—with hardware firmware deployment confirmed on physical hardware—and by subjecting this combination to a structured SITL-based verification methodology.

III. PROBLEM STATEMENT

Let G denote a GCS providing: (i) a MAVLink telemetry endpoint T ; (ii) a command interface C supporting a defined core command set; (iii) a human-readable browser interface U ; and (iv) a command-filtering safety layer S . Prior web-based GCS systems realise $G = \{T, C, U, S\}$ but require a general-purpose computer H (≥ 512 MB RAM) to execute $\{T, C, S\}$. DroneBridge realises only T on an ESP32.

Research question: Can a useful approximation of $G = \{T, C, U, S\}$ covering core interactive UAV operations be realised on a microcontroller M with $\text{RAM} \leq 327$ KB and no general-purpose OS, with command-stack functionality verified in SITL and hardware firmware confirmed deployable to physical hardware?

IV. SYSTEM DESIGN

A. Hardware Platform

Skylink targets the ESP32-WROOM-32 [10]: dual-core Xtensa LX6 at 240 MHz, 327 KB usable SRAM, 4 MB flash (1.31 MB application partition), and integrated 802.11 b/g/n Wi-Fi at 2.4 GHz, at approximately \$3–5 USD. An ESP32-WROOM-32 development board has been assembled as the GCS companion computer. In hardware mode, the firmware initialises `HardwareSerial2` on GPIO 16 (RX) and GPIO 17 (TX) at 115200 baud, 8N1, connecting to the Pixhawk 2.4.8 TELEM2 port.

Fig. 1 shows the complete hardware wiring diagram including the Pixhawk autopilot, four brushless motors with SimonK 30 A ESCs, the power distribution board (PDB), a 3S LiPo battery, a 5 V/3 A BEC, and the Ublox NEO-M8N GPS/compass module. This wiring has been assembled on physical hardware and the hardware firmware confirmed operational (Section VI-C); UART/Pixhawk MAVLink communication bench testing is the immediate next validation step.

B. Three-Tier Architecture

As illustrated in Fig. 2, Skylink follows a three-tier architecture:

Top tier (Client GCS Dashboard). A 4,347-line single-page application (HTML/CSS/JavaScript; 1,947 handwritten lines excluding the bundled Leaflet library; 147 KB Leaflet flash asset) served from LittleFS flash, rendered in any web browser. Four tabs—Map, Fly, Status, Log—provide the complete operator interface. The Map tab hosts a Leaflet.js interactive map with a yaw-rotated drone marker, home marker,

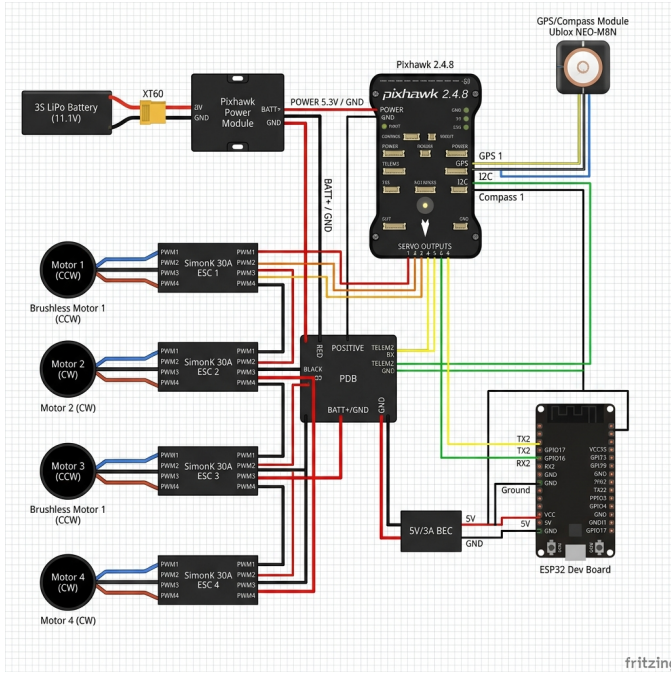


Fig. 1. Hardware wiring diagram of the Skylink UAV platform.

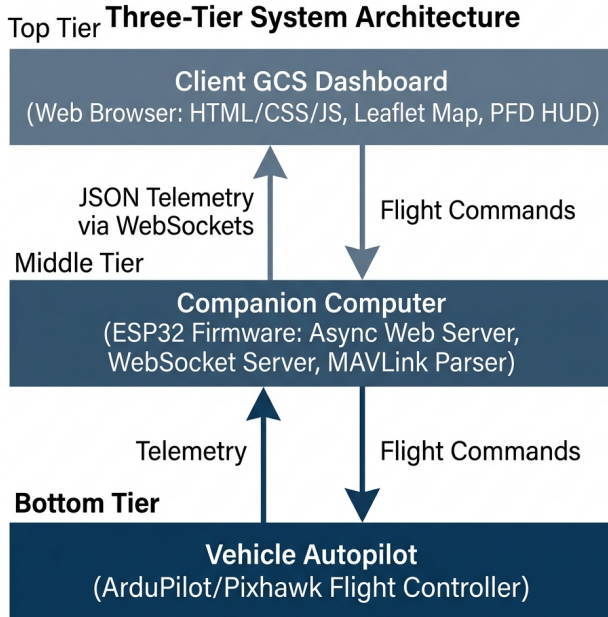


Fig. 2. Three-tier architecture of the Skylink ground control system.

a 60-point position trail, a 1000 m geofence circle, and a click-to-fly confirmation sheet. The Leaflet library is bundled offline, enabling map navigation without internet access. The dashboard also renders a Primary Flight Display (PFD) with artificial horizon and altitude tape. Fig. 3 shows a representative capture of the dashboard taken during SITL verification; the same single-page application is served from LittleFS by the physical board in hardware mode, since the top-tier asset bundle is identical in both compile-time configurations.

TABLE I
SKYLINK FIRMWARE MODULES. THE NINE MODULES LISTED HERE SUM TO 1,778 LOC; THE REPORTED TOTAL OF 1,893 INCLUDES `main.cpp` AND `setup.cpp` (≈ 115 LOC), SHOWN IN THE FINAL ROW.

Module	LOC	Role
FlightController	843	MAVLink v2 parse/dispatch, telemetry, mutex
WebServerModule	358	AsyncWebServer, WS dispatch, JSON
WiFiManager	173	Auto-connect, RSSI monitor
LedController	107	LED state machine (4 patterns)
TimeSync	67	NTP synchronisation
ConfigManager	66	EEPROM/Preferences
OTAUpdater	61	Wireless firmware update
Logger	54	Timestamped serial output
BuildInfo	49	FW/FS version matching
main.cpp (setup)	≈ 115	Initialisation, task creation
Total (.cpp)	1,893	
Total (.h)	730	

Middle tier (Companion Computer – ESP32). Firmware implementing an AsyncWebServer, WebSocket server, and MAVLink parser, running on FreeRTOS. The ESP32 acts simultaneously as HTTP server, WebSocket hub, and MAVLink protocol gateway. This firmware has been deployed to the physical ESP32 board and confirmed operational (Section VI-C).

Bottom tier (Vehicle Autopilot). A Pixhawk 2.4.8 running ArduCopter (hardware mode, assembled and awaiting UART bench test), or ArduCopter SITL over TCP (SITL mode, fully validated), communicating with the middle tier via MAVLink v2.

C. Firmware Module Structure

The firmware comprises nine C++ modules built with PlatformIO and the Arduino framework. Table I summarises each module's role and source-line count. The FlightController module (843 LOC) carries the largest share of complexity, implementing the MAVLink v2 parse-dispatch loop and the command-filtering layer.

D. Dual-Mode Compile-Time Architecture

A single compile flag (`-D SITL_MODE`) selects the MAVLink transport layer, as shown in Fig. 4. In SITL mode the FlightController opens a `WiFiClient` TCP connection to port 5763 on the PC running ArduCopter SITL. In hardware mode it uses `HardwareSerial2` on GPIO 16/17. All MAVLink logic, safety checks, and telemetry processing are identical in both modes; only the byte I/O layer differs. The `isSITLTcpConnected()` function returns `true` unconditionally in hardware mode, so the dashboard MAVLink-active indicator correctly reflects the physical UART connection state once a HEARTBEAT is received. This architecture produces two distinct firmware binaries—one for SITL (verified) and one for hardware (deployed and confirmed bootable; Pixhawk UART communication pending bench test). This design is the key enabler of SITL-first development for embedded GCS firmware.

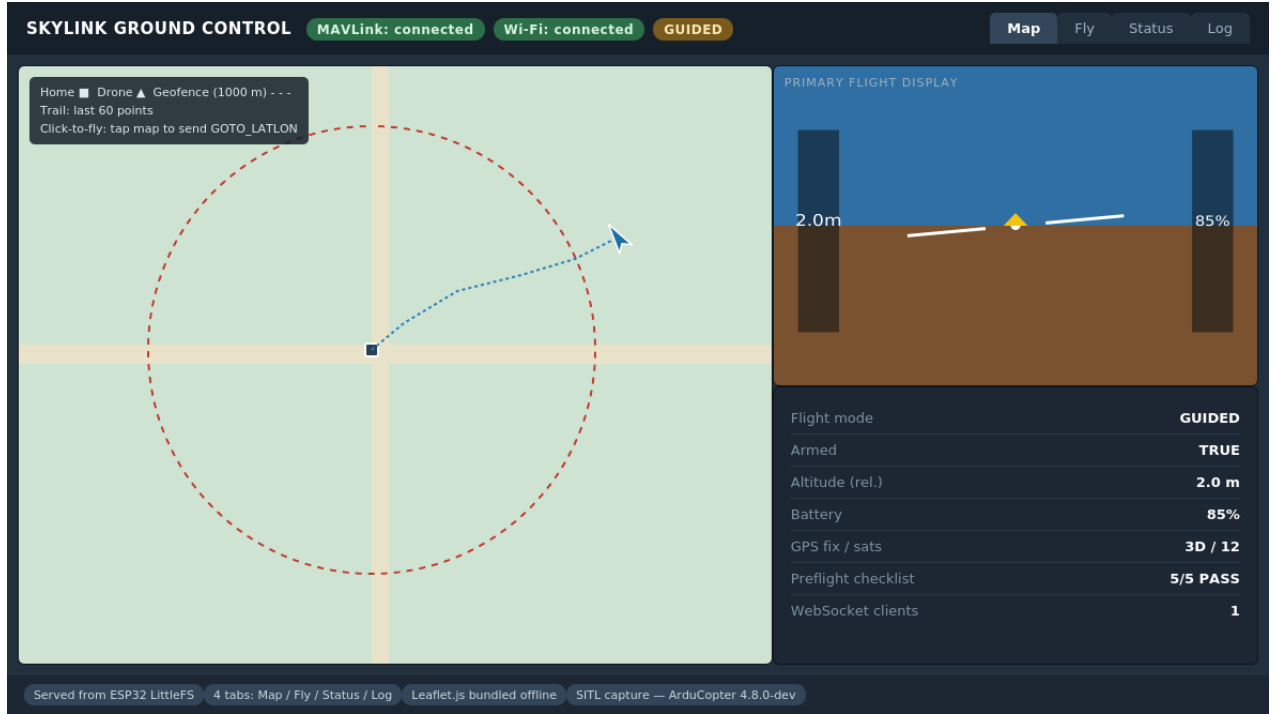


Fig. 3. Skylink browser dashboard (Map tab), captured during SITL verification: Leaflet.js map with home marker, live drone marker, 60-point position trail, and 1000 m geofence circle (left); Primary Flight Display with artificial horizon and altitude/battery tapes, and live status panel (right). The identical asset bundle is served from LittleFS flash in both SITL and hardware compile-time configurations.

Dual-Mode compile-time transport selection in firmware

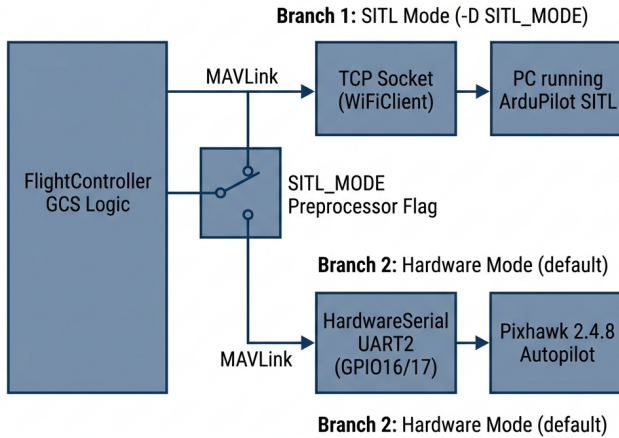


Fig. 4. Compile-time transport selection for SITL and hardware modes.

V. METHODS

A. MAVLink Protocol Implementation

Skylink uses the `okalachev/MAVLink Arduino` library (v2.0.29) [11]. Each received byte is fed into `mavlink_parse_char()` on `MAVLINK_COMM_0`; com-

TABLE II
MAVLink MESSAGE TYPES PARSED BY SKYLINK.

Message	Fields Extracted
HEARTBEAT	Armed state, flight mode
SYS_STATUS	Battery voltage, remaining %
ATTITUDE	Roll, pitch, yaw (rad → deg)
VFR_HUD	Altitude (MSL), groundspeed
GLOBAL_POSITION_INT	Lat, lon, rel. alt, velocity
GPS_RAW_INT	Satellites, fix type
HOME_POSITION	Home lat/lon
COMMAND_ACK	Command ID, result code
STATUSTEXT	Autopilot text (5-entry ring)

pleted frames are dispatched by message ID. Table II lists the nine message types handled, together with the fields extracted from each.

The GCS heartbeat (`MAV_TYPE_GCS`, system ID 255, component ID 190) is transmitted at 1 Hz. Data stream requests use `REQUEST_DATA_STREAM` (MAVLink legacy API, Message ID 66) targeting six stream groups (`ALL`, `RAW SENSORS`, `EXTRA1–EXTRA3`, `POSITION`) at 4 Hz; this interface is universally supported across ArduPilot firmware versions and is preferred over `MAV_CMD_SET_MESSAGE_INTERVAL`, which returns `UNSUPPORTED` on the target Pixhawk 2.4.8 firmware. All outgoing MAVLink packets use `sysid=255` as the source identifier to ensure consistent identity with the autopilot’s connection-tracking logic.

B. WebSocket Command Protocol

Commands are transmitted as JSON objects over the `/ws` WebSocket endpoint. Listing 1 shows representative

```
// Command (browser -> ESP32):
{"v":1,"type":"command","command":"TAKEOFF","altitude":5.0}
{"v":1,"type":"command","command":"MOVE_BODY",
 "x":3.0,"y":0.0,"z":0.0}

// ACK (ESP32 -> browser, forwarded from autopilot
  COMMAND_ACK):
{"v":1,"type":"event","event":"ACK",
 "command":22,"result":0,"result_name":"ACCEPTED","ok":true}

// Telemetry heartbeat (ESP32 -> browser, 5 Hz):
{"v":1,"type":"event","event":"HEARTBEAT",
 "armed":false,"flight_mode":4,"flight_mode_name":"GUIDED",
 "altitude":5.2,"relative_alt":5.1,
 "lat":-35.363,"lng":149.165,
 "battery_remaining":85,"gps_fix":3,"gps_sats":12,
 "mav_connected":true,"wifi_connected":true,"ws_connected":
  true}
```

Listing 1. WebSocket message formats. Commands: browser → ESP32. ACK and HEARTBEAT events: ESP32 → browser at 5 Hz.

command, acknowledgement, and telemetry heartbeat message formats. Commands flow from browser to ESP32; telemetry heartbeats flow from ESP32 to browser at 5 Hz. Command acknowledgements originate from the autopilot's COMMAND_ACK response and are forwarded as ACK events. Table III lists all fifteen dispatched commands with their preconditions and filtering locations.

C. Concurrency and Thread Safety

The firmware runs on FreeRTOS. A SemaphoreHandle_t fcMutex guards all access to the shared FCTelemetry struct between the main loop() task and AsyncWebServer ISR callbacks. Non-blocking takeMutex(0) is used in handle() to prevent starving the Wi-Fi stack. A single WebSocket client is enforced via ws.cleanupClients(1) on connect; multiple simultaneous clients would receive duplicate flight commands, creating a safety hazard in hardware mode.

D. Firmware-Level Command Filtering

Safety filtering is applied at the firmware layer, independently of autopilot parameters. The motion-command gate for MOVE_BODY is:

$$\text{canMove} = \underbrace{M}_{\text{MAVLink}} \wedge \underbrace{A}_{\text{Armed}} \wedge \underbrace{G}_{\text{GUIDED}} \wedge \underbrace{F}_{\text{GPS-3D}} \wedge \underbrace{(h \geq h_{\min})}_{\text{AGL check}} \quad (1)$$

where h is the current relative altitude and $h_{\min} = 2$ m. Equation (1) includes the armed predicate A , consistent with the implemented precondition set in Table III. The same canExecuteGuidedMoveUnlocked() precondition function is invoked by the GOTO_ALT dispatch path, so the AGL check and its associated fallback defect (Section VII-D) apply identically to both MOVE_BODY and GOTO_ALT.

Safety constants (Table IV) are compile-time values in skylink_config.h, requiring a firmware reflash to modify. This ensures that filtering thresholds cannot be relaxed by runtime configuration or operator input. Table VIII presents the command-level constraint coverage matrix, showing which predicates apply to each motion-affecting command and the SITL test status for each.

E. Phase-Gated Development Methodology

All features were implemented and acceptance-criteria verified in ArduCopter SITL before hardware assembly. Table V enumerates the five development phases and their acceptance criteria.

F. Experimental Setup

- **ESP32:** ESP32-WROOM-32 development board (physical hardware), PlatformIO v6.x, Arduino framework.
- **ArduPilot:** ArduCopter SITL [12] 4.8.0-dev, WSL2 Ubuntu 22.04, mirrored networking.
- **SITL ports:** TCP 5760 (MAVProxy), 5762 (Mission Planner observer), 5763 (Skylink).
- **Libraries:** ESPAsyncWebServer3.0.0 [13], ArduinoJson7.0.0 [14], MAVLink2.0.29 [11].
- **Build:** SITL validation used FW Build 12, FS Build 15 (24 May 2026). The hardware-mode firmware was subsequently deployed at FW Build 12 with FS Build 17 (current hardware build); FS Build 17 incorporates minor LittleFS filesystem corrections applied after SITL validation completed.
- **SITL validation scope:** All command-path results are from the SITL (TCP) path. The hardware-mode firmware has been deployed and confirmed operational on the physical ESP32 board; UART/Pixhawk MAVLink communication bench testing is the immediate next step.

VI. RESULTS

A. Memory Footprint

The central feasibility question for a resource-constrained microcontroller GCS is whether a complete protocol stack can coexist within 327 KB SRAM. Table VI presents the steady-state resource profile at Build 12 with one WebSocket client connected and MAVLink active in SITL mode.

ESP.getMinFreeHeap() records the minimum free heap observed since boot (the runtime heap low-water mark). A value of 51 KB during SITL operation demonstrates that the firmware stack retains meaningful dynamic heap headroom at peak load; the application binary occupied 860 KB of the 1,310 KB application partition (65.6%). Four architectural decisions contribute to heap efficiency: (1) static asset serving from LittleFS without RAM buffering; (2) a fixed-size FCTelemetry struct updated in-place; (3) a compile-time JSON buffer of 1 152 bytes; and (4) AsyncWebServer ISR processing without per-connection RAM allocation. These design choices are equally relevant to hardware-mode operation, as the firmware binary is architecturally identical between SITL and hardware builds.

B. SITL Functional Verification

All fifteen API commands and nine MAVLink message types were verified in ArduCopter 4.8.0 SITL before hardware assembly. Table VII presents the complete SITL verification matrix. Fig. 5(a) shows representative console output during a SITL TAKEOFF sequence. Every capability was verified in simulation; none of these results extend to the UART hardware path.

TABLE III

SKYLINK WEBSOCKET API: ALL FIFTEEN COMMANDS WITH PRECONDITIONS, FILTERING LOCATION, AND SITL VERIFICATION STATUS. COMMANDS MARKED † ARE SYSTEM UTILITIES. SAFETY CONSTRAINTS FOR MOTION-AFFECTING COMMANDS ARE DETAILED IN TABLE VIII. THE TAKEOFF ALTITUDE BOUND IS ENFORCED IN BROWSER JAVASCRIPT (1–50 m); THE FIRMWARE FORWARDS THE PARAMETER WITHOUT AN INDEPENDENT CAP.

Command	Preconditions / Notes	Filtering Location	SITL Verified
ARM_DRONE	MAVLink active	Firmware	✓
DISARM_DRONE	Always accepted	—	✓
SET_FLIGHT_MODE	Mode name string	Firmware	✓
TAKEOFF	MAVLink active; alt param; bound 1–50 m	Browser JS	✓
LAND	Sets LAND mode	—	✓
RTL	Sets RTL mode	—	✓
MOVE_BODY	GUIDED + armed + GPS-3D + AGL ≥ 2 m*; cap 200 m/axis	Firmware	✓
YAW_RELATIVE	GUIDED + armed + GPS-3D; $ \Delta\theta \leq 90^\circ$	Firmware	✓
GOTO_LATLON	Armed + GPS-3D + radius ≤ 1000 m; alt 2–50 m	Firmware	✓
GOTO_ALT	GUIDED + armed + GPS-3D + AGL ≥ 2 m*; alt 2–50 m	Firmware	✓
LOITER_HERE	Armed + GPS-3D; sets LOITER mode	Firmware	✓
RC_OVERRIDE	Raw RC channel values; no safety filtering†	None	✓
LED_SET†	Manual LED control	—	✓
LED_TOGGLE†	Toggle LED state	—	✓
PING†	Round-trip latency measurement	—	✓

*AGL floor has a known defect; see Section VII-D.

†RC_OVERRIDE may bypass higher-level safety rules; restrict to controlled SITL testing until a safety policy is defined.

TABLE IV

FIRMWARE SAFETY FILTER CONSTANTS (SKYLINK_CONFIG.H). ALL VALUES ARE COMPILE-TIME CONSTANTS. THE TAKEOFF ALTITUDE BOUND IS ENFORCED IN BROWSER JAVASCRIPT ONLY; THE FIRMWARE DOES NOT INDEPENDENTLY CAP THIS PARAMETER.

Parameter	Value
Altitude floor (AGL) for MOVE_BODY / GOTO_ALT*	2.0 m
MOVE_BODY per-axis displacement cap	200.0 m
GOTO_ALT / GOTO_LATLON altitude cap	50.0 m
Geofence radius (GOTO_LATLON)	1000 m
YAW_RELATIVE per-command maximum	90°
Command debounce interval	500 ms
Arm hold (UI)	1500 ms

*AGL floor has a known defect; see Section VII-D.

C. Hardware Firmware Deployment Verification

Following completion of the five SITL development phases, the hardware-mode firmware (compiled without SITL_MODE, targeting HardwareSerial2 on GPIO 16/17) was deployed to the physical ESP32-WROOM-32 development board and confirmed operational. Operational confirmation was established by observing:

- 1) The firmware initialisation log line [INFO] Initializing FlightController in [HARDWARE MODE] via UART2, confirming the correct binary was running on the physical device (FW Build 12, FS Build 17).
- 2) Successful Wi-Fi association, HTTP dashboard serving from LittleFS flash, and WebSocket connection establishment on the physical hardware—confirming the web-serving stack, LittleFS flash file system, and Wi-Fi driver operate correctly on-device.
- 3) LED state machine operation (four distinct blink patterns: Wi-Fi connecting, Wi-Fi connected, MAVLink active, armed), verified by direct observation of the physical board LED, confirming FreeRTOS task scheduling and the LED controller module operate on hardware.
- 4) Correct build-version verification at boot: [INFO]

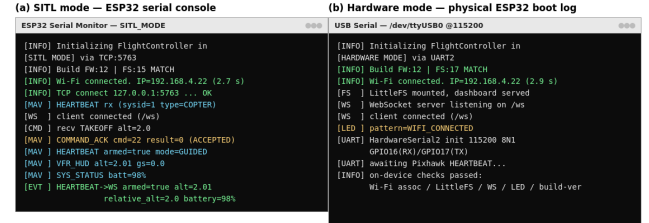


Fig. 5. ESP32 serial console output. (a) SITL mode: the ESP32 completes Wi-Fi connection, opens the SITL TCP transport, and receives a HEARTBEAT; a TAKEOFF command (alt 2.0 m) sent from the browser is acknowledged (result=0), and subsequent HEARTBEAT telemetry confirms the vehicle armed and airborne at 2.0 m with 98% battery. (b) Hardware mode: the boot log from the physical ESP32-WROOM-32 board, showing the [HARDWARE MODE] initialisation line, the Build FW:12 | FS:17 MATCH version check, Wi-Fi association, LittleFS dashboard serving, WebSocket client connection, and UART2 initialisation awaiting the Pixhawk HEARTBEAT.

Build FW:12 | FS:17 MATCH, confirming firmware and filesystem version integrity on the physical device.

An indoor motor-spin test (no propellers) has been identified as the next hardware-validation gate, verifying ESC arming, motor direction, and TAKEOFF command acceptance before any outdoor flight. UART/Pixhawk MAVLink communication bench testing—confirming telemetry parsing, command dispatch, and ACK handling over the physical UART connection—is the immediate subsequent step.

D. Comparison with Prior GCS Architectures

Table IX positions Skylink against the most relevant prior systems across key dimensions, including validation evidence level. Provenance for each row is given in the table caption rather than as a dedicated column, so that every remaining column is fully legible within the text width.

TABLE V
FIVE-PHASE SITL-FIRST DEVELOPMENT METHODOLOGY WITH ACCEPTANCE CRITERIA. ALL FIVE PHASES COMPLETED IN SITL; HARDWARE FIRMWARE SUBSEQUENTLY DEPLOYED TO PHYSICAL ESP32 AND CONFIRMED OPERATIONAL.

Phase	Objective	Acceptance Criterion
1	Transport + heartbeat	ESP32 connects to SITL TCP; receives HEARTBEAT within 5 s
2	Telemetry parse + dashboard	All 9 MAVLink types parsed; dashboard updates at 5 Hz
3	Command dispatch	ARM, TAKEOFF, LAND, RTL, MOVE_BODY issue correctly; COMMAND_ACK received
4	Map + click-to-fly	GOTO_LATLON triggered from map click; geofence blocks >1000 m
5	Command-filtering layer + LED	All Table IV constraints compile-time enforced; LED state machine functional

TABLE VI
SKYLINK RESOURCE PROFILE (ESP32, BUILD 12, SITL MODE).

Metric	Value	Reference	Method
Min. free heap (runtime)	51 KB	327 KB SRAM	ESP.getMinFreeHeap()
App flash usage	860 KB	1,310 KB partition	PlatformIO build
Flash utilisation	65.6%	1,310 KB partition	build report
Firmware LOC (.cpp)	1,893	—	cloc
Browser LOC (excl. Leaflet)	1,947	—	cloc
Leaflet.js asset	147 KB	—	file size

TABLE VII
SITL FUNCTIONAL VERIFICATION (ARДУСРTER 4.8.0-DEV, 24 MAY 2026, FW BUILD 12, FS BUILD 15). ALL RESULTS ARE SIMULATION-ONLY; HARDWARE-PATH PIXHAWK RESULTS ARE NOT REPORTED IN THIS WORK.

Capability	SITL Pass
Wi-Fi connect + dashboard load	✓
MAVLink link establishment (<5 s)	✓
9-message telemetry streaming (5 Hz)	✓
Preflight checklist evaluation	✓
GUIDED mode change	✓
Arm / Disarm	✓
Autonomous takeoff (5 m)	✓
Land / RTL	✓
MOVE_BODY (6 directions, 4 presets) [†]	✓
YAW_RELATIVE (45°/90°; 180° rejected) [‡]	✓
GOTO_LATLON (click-to-fly)	✓
Geofence block (>1000 m)	✓
LED state machine (4 states)	✓

[†]MOVE_BODY verified in SITL. AGL floor has a known defect; retesting required after fix before hardware flight.

[‡]YAW_RELATIVE: 45° and 90° executed; 180° exceeded the 90° cap and was correctly rejected (COMMAND_ACK result ≠ 0 confirmed).

VII. DISCUSSION

A. Feasibility of a Resource-Constrained Microcontroller GCS

The minimum free heap of 51 KB during SITL operation establishes a feasibility data point: a protocol stack encompassing telemetry parsing, command dispatch, Web-Socket serving, static file serving, and command filtering can coexist within 327 KB SRAM in a SITL-validated context. The subsequent deployment of the hardware firmware to a physical ESP32 board and confirmation of Wi-Fi, web-serving, and LED operation on-device provide a first real-hardware feasibility data point for this GCS architecture, independent of the SITL command-path validation. Prior web-based GCS work has consistently assumed that a general-purpose computer with at least 512 MB RAM is the minimum viable platform for protocol logic; Skylink challenges this

assumption, though UART/Pixhawk communication validation and controlled flight testing are needed to confirm the finding.

The cost implication at scale is notable. The per-unit GCS host-device cost is approximately \$3–5 (ESP32) versus \$15–35 (Raspberry Pi Zero 2W, as a representative companion computer). At 1000-unit deployment scale, this represents a hardware difference of approximately \$10 000–\$30 000. Beyond cost, the ESP32’s lower power draw (active Wi-Fi: approximately 240 mA at 3.3 V versus approximately 500 mA for Raspberry Pi Zero) and smaller physical footprint make it more suitable for weight-constrained UAV integrations. These figures are indicative; precise comparison would require a full bill-of-materials analysis.

B. SITL-First as an Embedded GCS Development Methodology

The dual-mode compile-time transport switch—a single `#ifdef` block in `FlightController.cpp`—means the transport layer is the only difference between the SITL-verified firmware and the hardware-deployed firmware. All GCS command and filtering logic can therefore be fully exercised against a simulated vehicle before any hardware is assembled. This ensures that the command and safety logic exercised during SITL testing is identical to the code running on the physical device, reducing hardware bring-up risk substantially. The SITL-first methodology thus directly supports the hardware deployment confidence described in Section VI-C, and provides a repeatable regression framework as features are added.

C. Comparison with Burke WebGCS

Burke [4] demonstrated a browser-accessible GCS on a Raspberry Pi, achieving arm/disarm, takeoff, land, RTL, real-time map tracking, and click-to-fly in real flight. Skylink implements a comparable set of core operator functions in SITL and has demonstrated hardware firmware deployment on physical hardware. The functional comparison cannot be stated as equivalence because Burke validated in real flight while Skylink has been validated only in SITL for command paths, and real-flight validation is pending. Skylink’s contribution is demonstrating this architecture on hardware approximately three orders of magnitude more RAM-constrained, at a lower host-device cost, with a preliminary command-filtering layer. A direct benchmark or user study against Mission Planner, QGroundControl, or Burke WebGCS is left to future work.

TABLE VIII
COMMAND-BY-CONSTRAINT SAFETY COVERAGE MATRIX.

Command	Armed	GUIDED	GPS-3D	AGL ≥ 2 m	Geofence	Cap
TAKEOFF	—	—	—	—	—	Browser-only
MOVE_BODY	Enforced	Enforced	Enforced	Defect*	Not applied	Enforced (200 m)
YAW_RELATIVE	Enforced	Enforced	Enforced	—	—	Enforced (90°)
GOTO_LATLON	Enforced	Auto [†]	Enforced	—	Enforced	Enforced (50 m alt)
GOTO_ALT	Enforced	Enforced	Enforced	Defect*	—	Enforced (50 m)
RC_OVERRIDE	—	—	—	—	—	None

*AGL floor defect: when `relative_alt` ≤ 0 (e.g., before home position is established), code falls back to MSL altitude, which may be non-zero at ground level, causing the 2 m floor to be bypassed. Both `MOVE_BODY` and `GOTO_ALT` call the same `canExecuteGuidedMoveUnlocked()` precondition function and are therefore both affected. Fix and retest before any hardware flight.

[†]`GOTO_LATLON` issues a mode-change command if not in `GUIDED`, rather than rejecting; `GUIDED` is not enforced as a hard precondition.

TABLE IX

COMPARISON OF WEB-BASED UAV GCS ARCHITECTURES. EVIDENCE LEVEL INDICATES THE HIGHEST-CONFIDENCE VALIDATION REPORTED. SKYLINK COMMAND-PATH RESULTS ARE SITL-ONLY; HARDWARE FIRMWARE DEPLOYMENT TO PHYSICAL ESP32 HAS BEEN CONFIRMED. BURKE WEBGCS [4] INCLUDES REAL-FLIGHT EVIDENCE. *Source*: SKYLINK – THIS WORK; BURKE WEBGCS – BURKE [4]; ATWAL DASHAPP – ATWAL [5], BACHELOR’S THESIS; IOT GCS – LESTARI ET AL. [6]; DRONEBRIDGE – OPEN-SOURCE PROJECT [7]; AEROGCS ENTERPRISE – COMMERCIAL PRODUCT [9].

System	Platform	Self-Hosted	API Cmds	Safety Layer	Security	Evidence Level
Skylink (this work)	ESP32, 327 KB	Yes	15 ^a	Firmware (partial ^b)	None	SITL + HW firmware deployed
Burke WebGCS [4]	RPi, 512 MB+	Yes	≈ 10	Python	Not stated	Real flight
Atwal DashApp [5]	PC + backend	No	> 10	DroneKit	n/a	Simulation
IoT GCS [6]	PC + backend	No	Few	None	n/a	100 m flights
DroneBridge [7]	ESP32, 327 KB	No (relay)	0	None	AES-GCM-256	Deployed
AeroGCS Ent. [9]	Cloud	Cloud-only	Many	Cloud	Cloud	Commercial

^a15 total API commands (Table III): 12 flight-related, 3 system utilities.

^bSafety constants implemented; AGL floor for `MOVE_BODY` and `GOTO_ALT` has a known defect (Section VII-D).

D. Limitations

SITL-only command-path validation. All reported command-path results are from the SITL (TCP) path. The UART pathway intended for Pixhawk hardware is implemented and the hardware firmware has been deployed and confirmed bootable; however, UART/Pixhawk MAVLink communication has not yet been bench-tested. No latency, jitter, packet-loss, RSSI, or range measurements are available.

Known AGL floor defect. In `canExecuteGuidedMoveUnlocked()`, when `telemetry.relative_alt` is zero or negative (e.g., before home position is established), the code falls back to MSL altitude (`telemetry.altitude`), which may be non-zero at ground level, causing the 2 m AGL floor to be bypassed. Because `GOTO_ALT` invokes the same precondition function as `MOVE_BODY`, both commands are affected. This defect must be corrected and retested in SITL before any hardware flight is attempted. The indoor motor-spin test procedure requires `ARMING_CHECK` = 0 to bypass GPS-related arming checks during bench validation; this setting must be restored to its operational value (`ARMING_CHECK` = 1) before any outdoor or flight testing.

TAKEOFF altitude bound enforced only in the browser. The TAKEOFF altitude parameter is clamped to 1–50 m in browser JavaScript only; the firmware forwards the parameter without an independent cap (Table III). Any WebSocket client speaking the documented JSON protocol could therefore bypass this bound. A firmware-side cap is planned as an immediate remediation item (Section VIII).

RC_OVERRIDE safety gap. The `RC_OVERRIDE` command passes raw RC channel values with no safety gate. This pathway may bypass higher-level safety rules and should be restricted to controlled SITL testing until an explicit safety policy is added.

Memory interpretation. `ESP.getMinFreeHeap()` returns the minimum free heap since boot, not an absolute measure of SRAM utilisation. Future revisions should distinguish free heap, minimum free heap, static RAM allocation, stack usage, and application-flash occupancy from the linker map.

No security. MAVLink message signing and WebSocket authentication are not implemented. The system exposes arming and navigation commands over an unauthenticated Wi-Fi channel, making the current implementation appropriate only for controlled laboratory or SITL testing.

Single WebSocket client. Multiple simultaneous clients are not supported; a second connection displaces the first.

No formal usability study. No evaluation against Mission Planner or QGroundControl has been conducted.

VIII. FUTURE WORK

Immediate priorities: (1) Fix the `moveBody()` AGL floor defect and retest all altitude-boundary cases (`MOVE_BODY` and `GOTO_ALT`) in SITL; (2) add an independent firmware-side cap on the TAKEOFF altitude parameter, removing reliance on the browser-only 1–50 m bound; (3) RSSI and range characterisation at 10–200 m.

Medium-term: (4) WebSocket authentication and MAVLink v2 message signing; (5) `RC_OVERRIDE` safety policy; (6) multi-client arbitration with command priority;

(7) mission upload in AUTO mode using the waypoint protocol; (8) formal usability study using NASA-TLX and SUS instruments; (9) correct memory profiling (static DRAM from linker map, peak stack depth).

Long-term: (10) natural-language control interface via the Model Context Protocol (MCP) as demonstrated by Ramos-Silva and Burke [8]; (11) offline map tile caching in LittleFS; (12) 4G/LTE beyond-line-of-sight relay using the Waveshare SIM7600G-H module.

IX. CONCLUSION

This paper presented Skylink—an ESP32-hosted, browser-based GCS prototype for ArduPilot UAV applications. The implementation combines a complete browser interface served from LittleFS flash, WebSocket telemetry exchange at 5 Hz, MAVLink v2 message parsing across nine message types, a PFD with artificial horizon and altitude tape, and fifteen API command dispatch on an ESP32-class microcontroller (327 KB SRAM, FreeRTOS, Arduino framework). A compile-time dual-mode transport abstraction enables the complete GCS firmware to be validated against ArduPilot SITL before hardware assembly. Five development phases with explicit acceptance criteria were completed and SITL-verified. A minimum runtime free heap of 51 KB and a flash occupancy of 860 KB (65.6% of the application partition) were observed under peak SITL load.

Following SITL validation, the hardware-mode firmware was deployed to and confirmed operational on a physical ESP32-WROOM-32 board, with Wi-Fi association, LittleFS-served dashboard, WebSocket communication, LED state machine, and build-version integrity all confirmed on-device. To the authors' knowledge, within the surveyed literature, this is the first system combining a self-hosted browser GCS, a firmware-level command-filtering layer, and a dual SITL/hardware compile-time design on a single ESP32-class device. The current evidence supports the feasibility of this architecture; UART/Pixhawk bench testing, complete command-safety coverage (including correction of the known `moveBody()` AGL floor defect), communication security, and controlled flight validation remain as the immediate next steps.

ACKNOWLEDGEMENTS

The authors thank Dr. Akshay Pandey for guidance and supervision throughout this project, and IIITDM Jabalpur for laboratory access. This work was carried out as a Bachelor of Technology project.

REFERENCES

- [1] ArduPilot Development Team, "ArduPilot: Open Source Autopilot," *ardupilot.org*, 2025. [Online]. Available: <https://ardupilot.org>
- [2] M. Osborne, "Mission Planner Ground Control Station," *ardupilot.org*, 2025. [Online]. Available: <https://ardupilot.org/planner>
- [3] QGroundControl Development Team, "QGroundControl," *docs.qgroundcontrol.com*, 2025. [Online]. Available: <https://docs.qgroundcontrol.com>
- [4] P. J. Burke, "AI generated drone command and control station hosted in the sky," *npj Artificial Intelligence*, vol. 2, Article 43, 2026. doi: 10.1038/s44387-026-00101-6
- [5] J. S. Atwal, "Drone control and monitoring by means of a web application," Bachelor's Degree Final Project, Universitat Politècnica de Catalunya (UPC), Sept. 2023.
- [6] P. E. W. Lestari, M. Ridwan, R. F. Ramadhan, and F. Nisfa, "Real-Time IoT-Integrated Ground Control Station (GCS) for Unmanned Aerial Vehicle (UAV) Monitoring System," *Journal of Electrical Engineering and Education*, vol. 9, no. 2, Oct. 2025. doi: 10.21070/jeeeu.v9i2.1710
- [7] D. Feger and contributors (DroneBridge), "DroneBridge for ESP32: A secure and transparent telemetry link," *GitHub*, 2019–2025. Available: <https://github.com/DroneBridge/ESP32>
- [8] J. N. Ramos-Silva and P. J. Burke, "A Universal Large Language Model–Drone Command and Control Interface," *arXiv:2601.15486*, 2026. doi: 10.48550/arXiv.2601.15486
- [9] PDRL, "AeroGCS Enterprise User Manual–Drone Control Center," *aerogcs.com*, 2025. [Online]. Available: <https://aerogcs.com>
- [10] Espressif Systems, "ESP32 Series Datasheet," v3.5, 2023. [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>
- [11] MAVLink Development Team, "MAVLink Micro Air Vehicle Communication Protocol," *mavlink.io*, 2025. [Online]. Available: <https://mavlink.io>
- [12] ArduPilot Development Team, "SITL – Software In The Loop Simulation," *ardupilot.org*, 2025. [Online]. Available: <https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html>
- [13] M. Carbou, "ESP Async WebServer," *GitHub*, 2024. Available: <https://github.com/mathieucarbou/ESPAsyncWebServer>
- [14] B. Fischler, "ArduinoJson: Efficient JSON library for embedded C++," *arduinojson.org*, 2024. Available: <https://arduinojson.org>